```
"""
Лабораторная работа № 1. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Итерационные методы вычисления элементарных функций:
    sqrt(a)  - метод Ньютона (формула Герона),
    cbrt(b)  - метод Ньютона,
    ln(c)    - разложение в степенной ряд.

Исходные данные варианта 13:
    a = 31, b = 100, c = 0.2, eps = 1e-5,
    критерий остановки - по разности соседних приближений,
    начальное приближение x0 = a / 2,
    дополнительное требование Д1 - таблица итераций для каждой функции
    и анализ изменения числа верных знаков.
"""
import math

A, B, C = 31, 100, 0.2
EPS = 1e-5
N_MAX = 100


# ---------------------------------------------------------------------------
# Вычислительные функции (math.sqrt, math.log, **0.5 внутри НЕ используются)
# Каждая функция возвращает пару (результат, число итераций).
# Если передан список trace, в него добавляются строки таблицы итераций
# (номер шага, приближение, |разность соседних приближений|).
# ---------------------------------------------------------------------------
def sqrt_newton(a, eps=EPS, n_max=N_MAX, trace=None):
    """Квадратный корень методом Ньютона (формула Герона), x0 = a / 2."""
    if not isinstance(a, (int, float)) or isinstance(a, bool):
        raise TypeError("Аргумент должен быть числом")
    if a < 0:
        raise ValueError("Подкоренное выражение должно быть неотрицательным")
    if a == 0:
        return 0.0, 0
    x = a / 2                                   # начальное приближение
    if trace is not None:
        trace.append((0, x, None))
    for n in range(1, n_max + 1):
        x_new = 0.5 * (x + a / x)               # итерационная формула
        diff = abs(x_new - x)
```

```
        if trace is not None:
            trace.append((n, x_new, diff))
        if diff < eps:                          # критерий по разности
            return x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def cbrt_newton(b, eps=EPS, n_max=N_MAX, trace=None):
    """Кубический корень методом Ньютона (с учётом знака), x0 = |b| / 2."""
    if not isinstance(b, (int, float)) or isinstance(b, bool):
        raise TypeError("Аргумент должен быть числом")
    if b == 0:
        return 0.0, 0
    sign = -1 if b < 0 else 1
    b = abs(b)
    x = b / 2                                   # начальное приближение
    if trace is not None:
        trace.append((0, sign * x, None))
    for n in range(1, n_max + 1):
        x_new = (2 * x + b / (x * x)) / 3       # итерационная формула
        diff = abs(x_new - x)
        if trace is not None:
            trace.append((n, sign * x_new, diff))
        if diff < eps:
            return sign * x_new, n
        x = x_new
    raise RuntimeError("Точность не достигнута за N_MAX итераций")


def ln_series(c, eps=EPS, n_max=10_000, trace=None):
    """Натуральный логарифм: ln c = 2(y + y^3/3 + y^5/5 + ...), y=(c-1)/(c+1)."""
    if not isinstance(c, (int, float)) or isinstance(c, bool):
        raise TypeError("Аргумент должен быть числом")
    if c <= 0:
        raise ValueError("Аргумент логарифма должен быть положительным")
    y = (c - 1) / (c + 1)
    y2 = y * y
    power = y                                   # текущая степень y^(2k+1)
    total = 0.0
    for k in range(n_max):
        term = 2 * power / (2 * k + 1)
        if trace is not None:
            trace.append((k, total, abs(term)))
        if abs(term) < eps:                     # критерий по модулю члена ряда
            return total, k
        total += term
```

```
        power *= y2
    raise RuntimeError("Точность не достигнута за n_max итераций")


# ---------------------------------------------------------------------------
# Вспомогательные функции вывода
# ---------------------------------------------------------------------------
def correct_digits(value, ref):
    """Число верных десятичных знаков после запятой: -lg|value - ref|."""
    err = abs(value - ref)
    if err == 0:
        return float("inf")
    return max(0.0, -math.log10(err))


def print_trace(title, trace, ref):
    print(f"\n{title}")
    print(f"{'Шаг':>4}{'x_n':>18}{'|x_n - x_(n-1)|':>20}"
          f"{'Погрешность':>16}{'Верных знаков':>15}")
    for n, x, diff in trace:
        d = "—" if diff is None else f"{diff:.3e}"
        err = abs(x - ref)
        print(f"{n:>4}{x:>18.10f}{d:>20}{err:>16.3e}"
              f"{correct_digits(x, ref):>15.2f}")


def main():
    tasks = [
        ("√a", "sqrt", sqrt_newton, A, math.sqrt(A)),
        ("∛b", "cbrt", cbrt_newton, B, B ** (1 / 3)),
        ("ln c", "ln", ln_series, C, math.log(C)),
    ]

    print("Лабораторная работа № 1, вариант 13")
    print(f"a = {A}, b = {B}, c = {C}, eps = {EPS}, x0 = a/2, критерий: по разнос

    # --- Таблица результатов (аналог табл. 3 из задания)
    print(f"\n{'Функция':<10}{'Аргумент':>10}{'Результат':>16}"
          f"{'Эталон':>16}{'Погрешность':>14}{'Итераций':>10}")
    traces = {}
    for label, name, f, arg, ref in tasks:
        tr = []
        val, n = f(arg, trace=tr)
        traces[name] = (tr, ref)
        print(f"{name:<10}{arg:>10}{val:>16.10f}{ref:>16.10f}"
              f"{abs(val - ref):>14.2e}{n:>10}")
```

```
    # --- Д1: таблицы итераций
    print("\n=== Д1. Таблицы итераций ===")
    titles = {"sqrt": f"Квадратный корень √{A} (x0 = a/2)",
              "cbrt": f"Кубический корень ∛{B} (x0 = b/2)",
              "ln": f"Натуральный логарифм ln {C} (частичные суммы ряда)"}
    for name in ("sqrt", "cbrt", "ln"):
        tr, ref = traces[name]
        print_trace(titles[name], tr, ref)

    # --- Д6-подобная проверка входных данных (не обязательна для варианта)
    print("\n=== Проверка обработки ошибочных данных ===")
    for f, arg in [(sqrt_newton, -4), (ln_series, 0), (cbrt_newton, "abc")]:
        try:
            f(arg)
        except (ValueError, TypeError) as e:
            print(f"{f.__name__}({arg!r}): {e}")


if __name__ == "__main__":
    main()
```